

LUCRARE DE LABORATOR NR. 7

RETELE NEURONALE HOPFIELD

1. SCOPUL LUCRARI

In aceasta lucrare se studiaza retelele neuronale Hopfield , implementandu-se o aplicatie tipica a acestora: memoria asociativa.

2.BREVIAR TEORETIC

Reteaua neuronală Hopfield constă din neuroni de tipul McCulloch Pitts, total interconectați. Fie N numărul acestor neuroni. Fiecare neuron are două stări posibile: $V=+1$ sau $V=-1$. Conexiunea între un neuron i și un neuron j este notată T_{ij} . Suma intrărilor unui neuron i este $I_i = \sum T_{ij}V_j$, iar starea rețelei ca întreg este dată de cele N valori V_i , fiind deci un cuvânt de N biți.

Fiecare neuron poate să-și schimbe starea la un moment aleator, asincron față de ceilalți neuroni, dar cu egală probabilitate. Putem spune că la fiecare impuls de ceas (imaginar), un neuron este selectat aleator să-și schimbe starea. Starea unui neuron este evaluată după regula lui McCulloch și Pitts:

$$V_i(t+1) = +1 \text{ dacă } \sum T_{ij}V_j \geq 0$$

$$V_i(t+1) = -1 \text{ dacă } \sum T_{ij}V_j < 0$$

Modelul Hopfield este recursiv, cu conexiuni bidirectionale între neuroni.

Retelele Hopfield sunt folosite ca memorii asociative și sunt folosite și în probleme de optimizare.

Ca memorie asociativă, problema este următoarea: să presupunem că rețeaua Hopfield este folosită pentru a memora un set de stări V_s , unde $s=1..n$. Aceste stări sunt denumite prototipuri.

În acest context, “a memora” înseamnă că toate cele V stări trebuie să fie stări stabile ale rețelei. În plus, ele trebuie să fie stări “atractori”, care permit ca fiecare stare stabilă să fie atinsă dintr-o stare ușor diferită.

Pentru a produce acest efect, Hopfield folosește conexiuni între neuronii rețelei, calculate după formula:

$$T_{ij} = \sum V_i V_j$$

$$T_{ii} = 0$$

Poate fi ușor demonstrat că aceste ponderi pot fi găsite aplicând regula lui Hebb. Regula lui Hebb presupune creșterea ponderii conexiunii între 2 neuroni, de fiecare dată când cei 2 neuroni sunt simultani activi. Într-o rețea Hopfield, aceasta se realizează practic, de următoarea manieră: rețeaua este inițializată cu un set complet nul de conexiuni, și ea este forțată într-o stare particulară V_s . Sunt examinate toate perechile posibile (i,j) , și ponderea T_{ij} este crescută cu ΔT_{ij} , unde ΔT_{ij} este calculată după următorul tabel:

V_i	V_j	ΔT_{ij}	
+1	+1	+1	neuroni simultan activi
+1	-1	-1	neuroni în opoziție
-1	+1	-1	neuroni în opoziție
-1	-1	+1	neuroni simultan inactivi

Se va vedea ca, dupa fiecare prezentare a unui exemplu de antrenare (prototip), valoarea ponderilor prin retea va deveni mai mare sau mai mica.

Pot fi posibile si formulari diferite ale regulii lui Hebb, unele din ele fiind mai plauzibile din punct de vedere biologic; in unele cazuri, daca cei 2 neuroni sunt simultan inactivi, ponderea conexiunii intre ei se poate lasa neschimbata, in loc sa fie crescuta ca in tabelul de mai sus.

Conform tabelului $\Delta T_{ij} = V_i V_j$.

In general, ponderea conexiunii (i,j) este o masura a corelatiei intre activarile neuronilor i si j , pe intregul set de prototipuri, care trebuie invatat.

Ca starea unui neuron sa fie stabila trebuie ca: $V_i(t+1) = V_i(t)$.

Dezvoltand aceasta expresie, se deduce o conditie suficienta pentru stabilitate : starile sa fie ortogonale una alteia (produsul scalar intre vectorul de stare din prototipul i si vectorul de stare din prototipul j este egal cu 0.)

Vedem ca regula lui Hebb, asigura stabilitatea cel putin pentru starile ce sunt ortogonale.

O notiune importanta in retelele Hopfield este cea de bazin de atractie. Bazinul de atractie al unei stari V este setul de stari V_s al retelei care evolueaza catre V dupa un numar finit de tranzitii. Pentru a obtine o imunitate maxima la zgomot, este necesar sa largim cat mai mult posibil acest bazin, in jurul starilor, folosite pentru a memora informatia.

In concluzie, modelul Hopfield difera mult de modelul MLP, si in arhitectura, si in procesul de invatare.

De remarcat ca invatarea in Hopfield este incrementala: pentru a adauga o noua stare stabila in retea, este necesar doar sa modificam ponderile T_{ij} , folosind regula lui Hebb.

Acest mod de calcul al ponderilor (Hebb), poate introduce in retea pe langa starile dorite si stari stabile suplimentare, parazite, nedorite. Acestea pot, in ultima instanta, sa fie atractori puternici. Au fost sugerate diverse solutii pentru a depasi problema. Una dintre solutii se bazeaza pe principiul : trial and error. Ponderile sunt modificate pentru a largi bazinul de atractie al starilor dorite si pentru a-l mica pe cel al starilor parazite. Aceasta se realizeaza positionind retea intr-o stare vecina a unei stari stabile, si permitind retelei sa se stabilizeze. Daca retea se stabilizeaza intr-o stare parazita, sau intr-o stare diferita de cea dorita, ponderile conexiunilor sunt modificate in consecinta. Acest proces se repeta pana cand toate starile vecine starilor stabile dorite, converg, evolueaza, de maniera dorita.

Retelele Hopfield permit implementarea memoriei asociative : o intrare, cum ar fi o forma (imagine sau un cuvint) incompleta sau partial acoperita de zgomot, este fortata in retea (adica toate iesirile neuronilor retele sunt fortati la valorile imaginii respective). Dinamica retelei (ce are ponderile calculate corespunzator), va converge catre o stare stabila, care in general, este starea dorita.

La o memorie asociativa, o problema care se pune este determinarea numarului de exemple (sabioane) ce pot fi stocate de retea. Rezultate teoretice, dar si numeroase simulari arata ca numarul optim de sabioane ce pot fi memorate este in jur de $0.14N$, unde N este numarul de neuroni din retea. Totusi, daca se incearca memorarea unui numar mai mare de sabioane decit $0.14N$, retea Hopfield prezinta o caracteristica foarte nedorita: uita catastrofic. O retea poate invata fara dificultate n exemple, si poate sa le uite pe toate cand mai trebuie sa invete un alt exemplu suplimentar.

3.DESFASURAREA LUCRARI

Se va realiza o aplicatie practica a retelelor neuronale Hopfield, simularea unei memorii asociative.

In aceasta aplicatie, daca prezentam un fragment dintr-o imagine binara pe intrarile retelei, aceasta se va stabili (va recunoaste) imaginea completa.

Reteaua Hopfield consta din 400 de neuroni. Vom memora in ea 20 de sabloane.

Primul sablon reprezinta o figura umana (o imagine usor de identificat ce reprezinta). Restul de 19 sabloane de memorat, sunt sabloane aleatoare, fiecare avind aproximativ 50% pixeli albi si 50% pixeli negri (aproximativ 200 de puncte albe si 200 de puncte negre).

Primul sablon (figura umana) este memorat intr-un fisier text: patt0.txt, si este creat manual, cu ajutorul editorului de texte edit din sistemul de operare DOS. Acest fisier se va salva in directorul c:\hopfield . Fisierul contine 400 de numere +1 si -1. (+1 se va reprezenta grafic ca un pixel negru. -1 se va reprezenta grafic ca un pixel alb.)

Continutul fisierului patt0.txt este urmatorul:

```
-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 -1 -1 -1 -1 1 1 1 1 -1 -1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 -1 1 1 1 1 1 1 1 1 -1 -1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 1 1 1 1 1 1 1 1 1 1 -1 -1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 1 1 1 1 1 1 1 -1 -1 -1 -1 -1 1 1 -1 -1 -1 -1
-1 -1 -1 1 1 1 1 1 1 1 -1 -1 -1 -1 -1 -1 -1 -1 1 -1 -1
-1 -1 1 1 1 1 1 1 1 1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 1 -1
-1 -1 1 -1 -1 1 1 1 -1 -1 1 -1 -1 -1 -1 1 -1 -1 1 -1
-1 1 1 -1 -1 1 1 1 -1 1 1 1 -1 -1 1 1 1 -1 -1 1
-1 1 1 1 -1 1 1 -1 -1 -1 1 -1 -1 -1 -1 1 -1 -1 -1 1
-1 1 1 1 -1 -1 1 -1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1 1
-1 -1 1 -1 -1 -1 1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1 -1 1
-1 -1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 1
1 1 1 1 1 1 1 1 -1 -1 -1 1 -1 -1 -1 1 -1 -1 1 1
1 1 1 1 1 1 1 1 1 -1 -1 -1 1 1 1 -1 -1 -1 1 1
1 1 1 1 1 1 1 1 1 -1 -1 -1 -1 -1 -1 -1 1 1 1
1 1 1 1 1 1 1 1 1 1 -1 -1 -1 -1 -1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

Celelalte 19 imagini 20x20 de memorat, sunt stocate pe disc, in 19 fisiere text: patt1.txt, patt2.txt, ..., patt19.txt care sunt generate automat prin program.

Programul dezvoltat calculeaza si memoreaza ponderile pentru care reseaua invata cele 20 de sabloane. Testarea memoriei asociative simulate se face "atacand" reseaua cu o imagine incompleta (figura umana incompleta). Aceasta imagine de test se stocheaza intr-un fisier text in directorul c:\hopfield si se creaza cu editorul edit.

Fisierul are urmatorul continut:

```
-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 -1 -1 -1 -1 1 1 1 1 -1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 -1 1 1 1 1 1 1 1 1 -1 -1 -1 -1 -1 -1
```

```
-1 -1 -1 -1 -1 1 1 1 1 1 1 1 1 1 -1 -1 -1 -1 -1
-1 -1 -1 -1 1 1 1 1 1 1 1 -1 -1 -1 -1 1 1 -1 -1 -1
-1 -1 -1 1 1 1 1 1 1 1 -1 -1 -1 -1 -1 -1 1 -1 -1
-1 -1 1 1 1 1 1 1 1 -1 -1 -1 -1 -1 -1 -1 -1 1 -1
-1 -1 1 -1 -1 1 1 1 -1 -1 1 -1 -1 -1 -1 1 -1 -1 1 -1
-1 1 1 -1 -1 1 1 1 -1 1 1 1 -1 -1 1 1 1 -1 -1 1
-1 1 1 1 -1 1 1 -1 -1 -1 1 -1 -1 -1 -1 1 -1 -1 -1 1
-1 1 1 1 -1 -1 1 -1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1 1
```

```
-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
```

Se va edita si apoi executa programul descris in continuare. Apoi in program se vor opera modificarile precizate in tema asociata programului.

Listingul programului

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <string.h>
#include <graphics.h>
#define WIDTH 10
#define HEIGHT 10
void drawGrafic(char * numeF);
void calculPonderi(void);
void calculToateIesiriRetea(void);
void drawGr(void);

//variabila globala : ponderile
//int w[400][400]; //Simetrie : w[j][i]=w[i][j], si w[i][i]=0; NU INCAPE !
int Y[20][20]; //iesirile calculate ale celor 20x20 neuroni
void main(void)
{
    char *numeF="c:\\hopfield\\patt0.txt";
    int nrPatt;
    int i,j;
    char numeCrt[81];
    char strNrCrt[7];
    FILE * fp;
    randomize();
    //construieste 19 fisiere 20x20 umplute aleator (cu -1 sau 1, uniform)
```

```
for(nrPatt=1;nrPatt<=19;nrPatt++){
    //calculeaza numele fisierului de deschis:
    itoa(nrPatt,strNrCrt,10);//converteste pe nrPatt (baza 10) la string
    strcpy(umeCrt,"C:\\hopfield\\patt");
    strcat(umeCrt,strNrCrt);
    strcat(umeCrt,".txt");
    printf("\n\n %s",umeCrt);
    getch();
    fp=fopen(umeCrt,"wt");
    if(fp==NULL){
        printf("\n Nu pot deschide fisierul nr. %d",nrPatt);
        getch();
        exit(1);}
    for(j=0;j<20;j++){
        fprintf(fp,"\n");
        for(i=0;i<20;i++)
            if(random(2)==0)fprintf(fp,"-1 ");
            else          fprintf(fp," 1 ");
        }//for j
        fclose(fp);
    }//for nrPatt

printf("\n Apasa o tasta pentru a desena patt0: ");getch();
drawGrafic("C:\\hopfield\\patt0.txt");
printf("\n Apasa o tasta pentru a desena patt1: ");getch();
drawGrafic("C:\\hopfield\\patt1.txt");
printf("\n Apasa o tasta pentru a desena patt2: ");getch();
drawGrafic("C:\\hopfield\\patt2.txt");
printf("\n Apasa o tasta pentru a desena patt3: ");getch();
drawGrafic("C:\\hopfield\\patt3.txt");
printf("\n Apasa o tasta pentru a desena patt19: ");getch();
drawGrafic("C:\\hopfield\\patt19.txt");

//initializeaza Y[20][20] cu sablonul incomplet:
fp=fopen("C:\\hopfield\\test.txt","rt");
if(fp==NULL){
    printf("\n Nu pot deschide fisierul test.txt ");
    getch();
    exit(1);}
//umple tabloul Y cu valorile din sablonul
//incomplet (memorat in test.txt):
for(j=0;j<20;j++)
    for(i=0;i<20;i++)
        fscanf(fp,"%d",&Y[j][i]);
//inchide fisierul deschis:
fclose(fp);
printf("\n Apasa o tasta pentru a desena imaginea de reconstituit (test.txt :)");
getch();
drawGrafic("C:\\referat1\\hopfield\\test3.txt");
```

```
clrscr();gotoxy(20,12);printf("Asteptati ! : calcul 400 iesiri retea !");
calculToateIesiriRetea();
printf("\nApasa orice tasta pt. a vedea imaginea calculata: ");
getch();
drawGr();
}
```

```
void drawGrafic(char * numeF)
{
    FILE * fp;
    int i,j;
    int patt[20][20];
    int gd=DETECT,gm;
    int xSus,ySus,xJos,yJos;

    initgraph(&gd,&gm,"C:\\borlandc\\bgi");
    fp=fopen(numeF,"rt");
    //umple patt:
    for(j=0;j<20;j++)
        for(i=0;i<20;i++)
            fscanf(fp,"%d",&patt[j][i]);
    //afisare pe ecran:
    setfillstyle(SOLID_FILL,WHITE);//pe acest monitor 286,
                                    // scrie de fapt cu BLACK

    for(j=0;j<20;j++)
        for(i=0;i<20;i++)
            if(patt[j][i]==1){
                xSus=i*WIDTH;
                ySus=j*HEIGHT;
                xJos=xSus+WIDTH;
                yJos=ySus+HEIGHT;
                bar(xSus,ySus,xJos,yJos);}//Daca era 0 :nu face nimic, lasa
                                    //ecranul alb

    fclose(fp);
    getch();
    closegraph();
}
```

```
void calculPonderi(void)
{
    char *numeF="c:\\hopfield\\patt0.txt";
    int nrPatt;
    int i,j;
    char numeCrt[81];
    char strNrCrt[7];
    FILE * fp;
    int patt[20][20][20];
    int linI,colI;
    int linN,colN;
```

```
int nrNeuron;
int w[400];
// int w[400][400]; Eroare : size too big !
//ponderile le memoram intr-un fisier text de numere
for(nrPatt=0;nrPatt<=19;nrPatt++){
    //calculeaza numele fisierului sablon curent de deschis:
    itoa(nrPatt,strNrCrt,10);//converteste pe nrPatt (baza 10) la string
    strcpy(umeCrt,"C:\\hopfield\\patt");
    strcat(umeCrt,strNrCrt);
    strcat(umeCrt,".txt");
    printf("\n\n %s",umeCrt);
    getch();
    fp=fopen(umeCrt,"rt");
    if(fp==NULL){
        printf("\n Nu pot deschide fisierul nr. %d",nrPatt);
        getch();
        exit(1);}
    //umple tabloul patt cu sablonul curent:
    for(j=0;j<20;j++)
        for(i=0;i<20;i++){
            fscanf(fp,"%d",&patt[nrPatt][j][i]);
        }
    //inchide fisierul deschis:
    fclose(fp);
}
//for nrPatt
//afisare patt:
printf("\n Afisare patt[0][][ ] : ");
for(j=0;j<20;j++){
    printf("\n");
    for(i=0;i<20;i++)printf("%d ",patt[0][j][i]);
}
//for j
//calcul ponderi:
//Deschidem fisierul text in care se salveaza ponderile:
fp=fopen("C:\\referat1\\hopfield\\ponderi.txt","wt");
if(fp==NULL){
    printf("Nu pot deschide fisier ponderi.txt !");
    getch();
    exit(1);}
for(nrNeuron=0;nrNeuron<400;nrNeuron++){

    gotoxy(1,1);printf("%d",nrNeuron);
    //Calcul ponderi w[i]:
    //w[i],i=0..399 este ponderea conexiunii intre nrNeuron si neuronul i
    for(i=0;i<400;i++){
        if(i==nrNeuron)w[i]=0;
        else{
            //este suma pentru cele 20 patternuri de produse iesiri nrNeuron si i:
            w[i]=0;
            for(nrPatt=0;nrPatt<20;nrPatt++){
                //nrNeuron se gaseste in linia linN si coloana colN:
```

```
linN=nrNeuron / 20;
colN=nrNeuron % 20;
//neuronul i se gaseste in linia linI si coloana colI:
linI=i / 20;
colI=i % 20;
w[i]=w[i]+patt[nrPatt][linN][colN]*patt[nrPatt][linI][colI];}for
} //else
//A calculat w[i] i=0..399.

//Scrie in fisier cele 400 de ponderi, cite 20 pe linie:
for(j=0;j<20;j++){
    fprintf(fp,"\n");
    for(i=0;i<20;i++)
        fprintf(fp,"%d ",w[20*j+i]);
} //for j
//A scris cele 400 de ponderi.
//Lasa mai multe linii goale , fata de urmatoarele 400:
fprintf(fp,"\n\n\n");
} //for nrPatt=0..19
fclose(fp); //a inchis fisierul text de ponderi
getch();
}

void calculToateIesiriRetea(void)
{
    int i,j,k;
    int nrNeuron;
    FILE * fp;
    long int xTotal;
    int w[400]; //cele 400 de ponderi intre neuronul crt si toti cei 400 din retea
    //initial iesirile sunt cele fixate de fisierul test.txt:
    //deschide fisierul de ponderi, pt. citire:
    fp=fopen("C:\\hopfield\\ponderi.txt","rt");
    if(fp==NULL){
        printf("\n Nu pot deschide fisierul ponderi.txt ");
        getch();
        exit(1);}
    //in ordine, de la neuron 0, la neuronul 399, se calculeaza asincron iesirile:
    for(nrNeuron=0;nrNeuron<400;nrNeuron++){
        gotoxy(1,1);printf("%d",nrNeuron);
        for(k=0;k<400;k++){
            //citeste w[k], ponderea conexiunii intre neuronul nrNeuron si neuronul k, din
            // fisierul ponderi.txt:
            //Se gaseste ca articolul numarul 400*nrNeuron +k, fata de inceputul fisierului
            // text ! Se vede ca se citeste in ordine, de aceea e simplu !
            fscanf(fp,"%d",&w[k]);} //for k

        //calcul xTotal, intrarea globala in nrNeuron, suma de produse XiWi:
        xTotal=0;
```



```
for(k=0;k<400;k++)xTotal=xTotal+w[k]*Y[k/20][k%20];
//reactualizeaza Y pentru neuronul curent: nrNeuron:
if(xTotal>=0)Y[nrNeuron/20][nrNeuron%20]=1;//ref: NCSOM pag. 28
else Y[nrNeuron/20][nrNeuron%20]=-1;
} //for nrNeuron
//A completat tot tabloul Y[20][20] !
//Inchide fisier ponderi:
fclose(fp);
} //calculToateIesiriRetea

void drawGr(void)
{
FILE * fp;
int i,j;
int gd=DETECT,gm;
int xSus,ySus,xJos,yJos;

initgraph(&gd,&gm,"C:\\borlandc\\bgi");
//afisare pe ecran:
setfillstyle(SOLID_FILL,WHITE);
for(j=0;j<20;j++)
for(i=0;i<20;i++)
if(Y[j][i]==1){
xSus=i*WIDTH;
ySus=j*HEIGHT;
xJos=xSus+WIDTH;
yJos=ySus+HEIGHT;
bar(xSus,ySus,xJos,yJos);} //if Daca era 0 :nu face nimic, lasa ecranul alb
getch();
closegraph();
}
```

Tema

-Sa se verifice cum functioneaza memoria asociativa daca editam alta imagine de baza (in locul figurii umane se va edita o alta imagine binara).

-Sa se creeze si alte fisiere text cu care sa se testeze reteaua.

4. CONCLUZII

In aceasta lucrare s-a studiat o aplicatie tipica a retelelor neuronale Hopfield: memoria asociativa. Experimental s-a constatat ca reseaua simulata recunoaste imaginea completa pe baza fragmentului de imagine aplicat pe intrarile ei (reseaua Hopfield s-a stabilizat pe imaginea completa).